Data Hub

User Manual

Application version 0.1.0

A desktop application for exploring data and building scientific charts

Contents

Introduction

Data Hub is a desktop application for Windows, macOS and Linux that lets you import tabular data, query it with SQL, and turn it into scientific charts — from histograms to 3D surfaces — without writing code.

Every project is a single file with the extension `.dhub`: a SQLite database that holds both the imported data and the definitions of every chart (figures, axes, series and their drawing options). The file is therefore self-contained and portable — moving or sharing it carries both the data and every visualization built on top of it.

Who this manual is for

This manual describes the application as it presents itself to a user: the main window, importing data, building queries, creating and customizing charts, running statistical operations on series, and the application's settings. A final chapter, Section 12, is aimed at developers who want to extend Data Hub with a new chart type or a new analysis operation.

Key concepts

- **Database (`.dhub`)**: the project file. Holds data tables and figures.
- **Table**: a set of data, either imported (from CSV, Excel, …) or produced by a saved query.
- **Saved query**: a named SQL statement, usable anywhere a table is.
- **Figure**: one tab in the chart panel; can hold one or more axes.
- **Axis**: a single chart inside a figure, with a chart type (histogram, scatter, …) and one or more series.
- **Series**: the data drawn on an axis, defined by a SQL query that produces the columns (**roles**) the chart type requires (e.g. `x`, `y`).
- **Renderer**: the piece of code behind a chart type — it turns a series' data into a Matplotlib drawing. See Section 6.1.

Starting up and managing databases

On first launch, or whenever no file is given, Data Hub asks which database to open:

New creates an empty `.dhub` database at a path you choose.
Open opens an existing `.dhub` file.
Create demo builds one of the shipped demo projects and opens it immediately — useful for exploring the application without your own data.

The last database opened is remembered and offered again on the next launch.

Saving and duplicating a project

A SQLite database writes its own changes straight to disk on every operation, so there is no separate "Save" for data. Two related commands are available instead:

Save As... saves the current database under a new name/path and switches to working on that copy, leaving the original untouched.
Optimize DB checks the database for problems, reports them, and compacts it (`VACUUM`, `ANALYZE`) to shrink it on disk and speed up opening it.

The main window

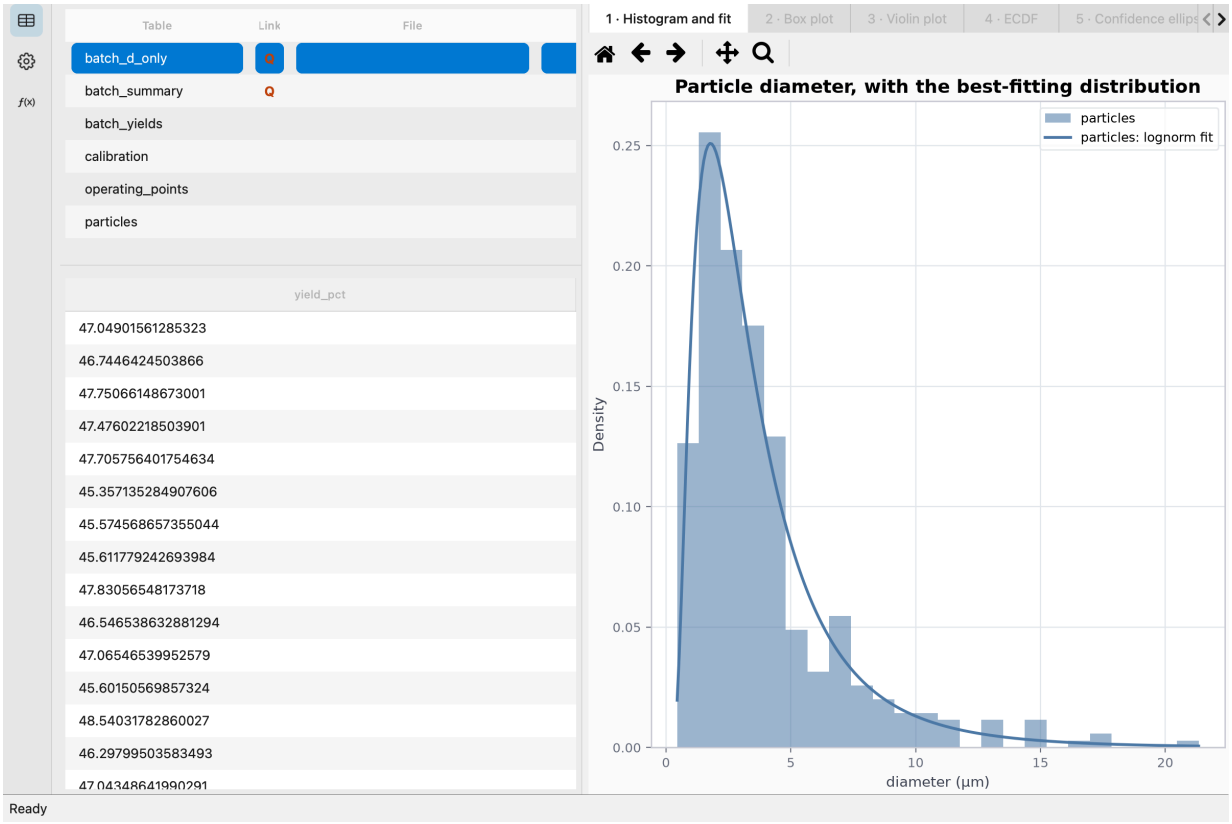

Figure 1: The main window: table list and data preview on the left, chart panel on the right.

The main window splits into two areas.

Navigation rail (left)

A vertical column of icons switches between the application's main sections:

- **Data** — the list of tables and queries in the database, with a data preview.
- **Chart Options** — properties of the currently selected figure, axis and series.
- **Series Operations** — analysis tools (fit, statistics, filtering, …) that apply to a series' data.
- **Menu** — opens the application's main menu (new, open, import, settings, credits, …).

Data panel

Lists the tables in the database (saved queries are marked with a **Q** icon) and, once one is selected, shows a preview with its first rows and columns. This is also where importing data and opening the Query Builder start.

Chart panel

Occupies the right side of the window and is organized into tabs — each tab is a **figure**. The toolbar above the chart offers Matplotlib's usual navigation tools:

Home	Return the chart to its initial view

Arrows ←/→	Step back/forward through the zoom history
Pan	Drag to move the visible area
Zoom	Draw a rectangle to zoom into an area

Zoom to fit brings the whole chart back within the panel's bounds in one click.

The **New plot** button on the Data page creates a new, empty figure, configured as described in Section 6.

Importing data

Import opens a dialog with four ways to bring data in:

Open	A local file: `.csv`, `.tsv`, `.txt`, `.xlsx`, `.xlsm`, `.xls`, `.json`, `.xml`.
Paste	Whatever tabular text is currently on the clipboard.
Database	One table read out of another database — SQLite (including another `.dhub` file), PostgreSQL, or MySQL.
Web	Whatever an http(s) URL returns — a CSV export, a JSON API, a spreadsheet.

Whichever way the data arrives, the same preview, column-type mapping and destination-table name apply before anything is written. For Excel sources with more than one sheet, the dialog asks which one to import.

Database opens its own small window: pick an engine (SQLite file, PostgreSQL or MySQL), fill in a file path or a host/port/database/username/password, and press **Connect** to list that database's own tables — never any internal bookkeeping tables Data Hub itself may have added to it, for a SQLite/`.dhub` source. Pick one and confirm to bring it back to the import dialog like any other source.

> An imported table stays available to every query and chart in the project: importing is a one-time read, not a live link back to the original file, database or URL — the data does not change on its own afterwards. Every source **except** a paste can be re-read later, though: right-click the table and choose **Update link** to replace its contents with a fresh read from the same file, database table, or URL.

> A PostgreSQL or MySQL connection's password is never saved — not in the project file, not anywhere on disk. **Update link** asks for it again each time, because a `.dhub` project is exactly the kind of file that gets copied, emailed or committed without a second thought, and a password sitting in plain JSON inside it would travel right along.

> **Web** only fetches `http://` and `https://` URLs — never a local path — so pasting a link here cannot read a file off disk by way of a `file://` address.

Exporting data

The content of a table or the result of a query can be exported with:

Export CSV writes the data to a `.csv` text file.
Export Excel writes the data to an `.xlsx` workbook.

Query Builder

The **Query Builder** (menu **Query Builder**) writes, checks and saves SQL statements that can be used as tables — handy for filtering, joining or aggregating imported data without duplicating it.

The editor has a **Run** button to check the result before saving, and a set of ready-made snippets that insert the right SQL skeleton:

Select	`SELECT` over a chosen table
Filter	`WHERE` with the comparisons spelled out
Join	`LEFT JOIN` keeping every row of the chosen table
Order	`ORDER BY` with an explicit direction
Summary	Count, average, min and max, grouped by a column
Union	`UNION ALL` of two tables with the same columns

A saved query appears in the table list with a **Q** icon and can be used as the data source for any series, exactly like an imported table. Unlike a table, its content is computed on the fly rather than stored: if the source data changes, the query returns fresh results the next time it runs.

Creating a chart

New plot opens the chart-creation window: pick a **chart type** (a renderer) and define the first series — the SQL query that supplies the data.

Every chart type requires the query to produce columns with specific names, its **roles** — for instance `x` and `y` for a scatter plot. The window shows the roles a type requires and a link to that type's Matplotlib documentation.

Chart types

Data Hub ships 23 chart types (**renderers**), grouped into the same categories Matplotlib's own documentation uses. Each is a self-contained piece of code that receives the rows a series' SQL query returns and draws them; see Section 12.1 for how a new one is added.

Pairwise data (x, y)

Bar Chart	Vertical bar chart
Horizontal Bar Chart	Horizontal bar chart
Broken Bar	Interval bars grouped by category (Gantt-style)
Broken Bar (Vertical)	The same, stacked in columns
Scatter Plot	Scatter plot
Fill Between	The region between two y curves over a shared x — confidence bands, tolerance limits
Stack Plot	Series stacked into filled bands, showing how a total divides into its parts
Table	A data table, rows and columns of text rather than a plot
Text	Text labels at data points, spread apart to avoid overlap
Time Series	Time series, with numeric or timestamp x
Timeline	Dated events on a baseline, each on its own stem — release histories, event lists
Wind Barbs	A barb per point showing the direction and strength of a vector field (u/v components)

Statistical distributions

Histogram	Histogram, supports multiple datasets
Box Plot	Box-and-whisker plot
Violin Plot	Estimated distribution of one or more samples
ECDF	Empirical cumulative distribution function
Pareto Chart	Categories sorted by descending magnitude with a cumulative-percentage line
Pie Chart	Pie or donut chart of one series
Stairs	A step outline over bin edges, for values that hold constant between them

Gridded / irregularly gridded data

Contour Plot	Filled or line contours of z over a regular x/y grid
Contour Plot (Scattered)	The same, for scattered (non-gridded) x/y/z data

3D and volumetric data

Surface Plot	3D surface over a regular x/y grid
Surface Plot (Scattered)	3D triangulated surface for scattered (non-gridded) x/y/z data

How a renderer reads a series

A renderer never sees your raw table — it sees the result of the series' SQL query, aliased to its required **roles**. A Scatter Plot needs `x` and `y`, so its series query is written as:

```
SELECT pressure AS x, flow AS y FROM operating_points
```

Optional roles work the same way and unlock extra behaviour when present — for instance `color` and `size` on a Scatter Plot drive a colour map and marker sizing, and `yerr`/`yerr_low`/`yerr_high` add error bars. The chart-creation window and the Chart Options panel both list which roles a given type accepts.

Every renderer also splits its adjustable settings into two families, both editable from Chart Options:

- **Plot arguments** (forwarded to Matplotlib as-is — line width, marker, transparency, colormap, …).
- **Options** (consumed by the renderer itself and never forwarded to Matplotlib — whether to draw a legend, how many bins, which fit to overlay, …).

Adding more series or axes

A figure can hold several axes (each drawn side by side or overlaid within the same tab), and each axis can hold several series, up to the maximum the chart type allows — some types, like Pie Chart, accept only one series; the application then proposes a new axis instead of a second series that would be dropped.

Add series in the Chart Options panel adds a new series to the selected axis.

Customizing a chart

From Chart Options, depending on the selected level:

- **Figure**: title, layout of its axes.
- **Axis**: title, axis labels, grid, legend, and the chart type's own options (e.g. the number of bins for a histogram).
- **Series**: the source query, colour, line or marker style, and any Matplotlib arguments forwarded straight to the drawing call (line width, transparency, …).

Every setting shows a contextual description, so reading Matplotlib's own documentation is rarely necessary to understand what a parameter does.

Series operations

Series Operations applies statistical or mathematical transformations to an existing series' data, previewing the result before it is committed to the chart. Every operation dialog shares the same layout: on the left, the source axis/series, a model and its parameters; on the right, a preview of the result and a log of the computation.

Fit	Fit data models	Fits a mathematical model (Gaussian, exponential, polynomial, a user function, …) to a series by least squares, and adds the fitted curve alongside the data.
Calculus	Differentiate or integrate	Computes the numerical derivative or the cumulative integral of a series.
Interpolate	Fill missing values	Fills gaps in a series (linear, spline, nearest, …), producing a complete curve from a sparse one.
Smoothing	Reduce noise	Applies a smoothing model (moving average, Savitzky-Golay, …) to reduce noise while preserving the underlying shape.
Outlier	Detect anomalies	Flags points that deviate from the rest of a series by a chosen statistical criterion.
Peaks	Find and measure peaks	Detects peaks in a series and reports their position, height and width.
Spectral	Analyse frequencies	Computes the frequency-domain content of a series (FFT-based).
Control Chart	Monitor process stability	Builds a statistical control chart (mean and control limits) and flags rule violations.
Cluster	Group similar data	Groups a series' points into clusters (k-means, DBSCAN, …) and labels each point by cluster.
Statistics	Compute metrics	Reports summary statistics (mean, standard deviation, quantiles, …) for a series.

A typical workflow is: select the axis and series to operate on, choose a model and its parameters, click **Preview** to see the result superimposed on the chart, adjust the parameters if needed, then confirm to add the result as a new series (or table) permanently. See Section 12.2 for how a new operation is added.

Appearance and styles

Matplotlib styles

Edit Matplotlib styles lets you adjust the base graphical parameters (rcParams) that govern the look of every chart — default colours, fonts, line widths. From this window you can:

- Load an existing `.mplstyle` file into the editor.
- Add an rcParam property to the editor.
- Reset a property to its Matplotlib default.
- Save the changes as a new `.mplstyle` file.

Application settings

Settings gathers the general preferences:

App style the look of the interface controls. Includes the platform's native styles plus, when installed, extra Qt styles (e.g. Breeze, Oxygen, QtCurve).

Language the interface language. Besides Auto (which follows the operating system's language), Italian is available.

Application log

The **Log viewer** shows the history of the application's internal operations (startup, opening a database, errors) — useful for diagnosing unexpected behaviour or attaching details to a bug report.

Demo projects

Create demo builds a pre-filled `.dhub` database with sample tables and a set of figures already configured across several chart types. It is the fastest way to explore the application's features without preparing your own data, and a good place to copy a series' or an axis' settings from into a real project.

Credits

Credits lists the application, the version in use, and the open-source libraries it is built on (including Qt/PySide6, Matplotlib, NumPy, pandas, SciPy and statsmodels).

Advanced: extending Data Hub

Data Hub discovers chart types and series operations the same way: by scanning a folder for Python classes that directly subclass a known base class, at import time — no registration list to edit and keep in sync. Dropping a well-formed file into the right folder is enough for it to appear in the application.

Writing a custom chart renderer

A chart type lives entirely in one file under `app/charts/`, as a class that subclasses `BaseAxisRenderer` (from `app.charts.base`) **directly** — the discovery scanner in `app/scanners/axis_renderer_scanner.py` matches that exact base-class name in the source, even when the class also inherits behaviour from another renderer.

The class is described entirely through its own attributes:

`Name`	Display name, and the `chart_type` value stored in the database. Renaming it orphans existing axes unless an alias is added to `CHART_TYPE_ALIASES` in the scanner.
`Category`	Which family of plot this is, following Matplotlib's own taxonomy (e.g. "Pairwise data", "Statistical distributions").
`Description` / `Link`	Shown in the chart picker and in the axis properties panel.
`RequiredRoles` / `OptionalRoles`	Column names the series' SQL query must (or may) produce — see Section 6.1.
`Kwargs`	Matplotlib keyword arguments forwarded verbatim to the plot call, as `{name: metadata}`. The metadata (`default`, `type`, `min`/`max`, `kind`, `group`, `description`) drives the generated editor UI automatically.
`Options`	Settings the renderer consumes itself and never forwards to Matplotlib — same metadata shape, opposite destination.
`MaxSeries`	How many series this renderer can draw on one axis, or `None` for any number.

The class then implements one method:

```
def render_axis(self, ax, series: list[SeriesData], options: dict | None = None) -> None:
    ...
```

which receives one `SeriesData` per series already queried into a DataFrame, and draws them onto `ax` (a Matplotlib `Axes`).

Practical steps:

1. Create `app/charts/my_chart.py`.
2. Import `BaseAxisRenderer` and, if useful, `SeriesData` from `app.charts.base`.
3. Define a class, e.g. `MyChartAxisRenderer(BaseAxisRenderer)`, setting `Name`, `Category`, `Description`, `RequiredRoles`/`OptionalRoles`, and optionally `Kwargs`/`Options`.
4. Implement `render_axis` using ordinary Matplotlib calls on `ax`.

5. Restart the application (or reopen the chart-creation window) — the new type appears in the picker with no further wiring.

`app/charts/text.py` is a good, compact, complete example to read or copy as a starting point: a handful of roles, a short `Kwargs` block, and a straightforward `render_axis`.

Writing a custom series operation

A series operation lives in `app/series_operations/`, as a class that subclasses `SeriesOperationDialogBase` (from `app.series_operations.dialog_base`) **directly** — discovered the same way, by `app/scanners/series_operation_scanner.py`.

The base class supplies the whole dialog shell (series picker, parameter form, preview pane, action buttons) and the plumbing that turns a result into a new series or table on the chart. A subclass sets a few class attributes —

`Name`	Display name shown in the Series Operations list.
`Description`	One-line summary shown next to the name.
`Icon`	An inline SVG source string for the operation's icon.

— and overrides the hooks the base class calls at the right moments. The ones every operation implements are:

`build_parameter_selector()`	Build the parameter form. Simple operations describe their parameters declaratively with the helpers in `app.series_operations.parameter_spec` (`FloatParam`, `IntParam`, `ChoiceParam`, …) rather than laying out widgets by hand.
`compute_results()`	Run the actual computation over the selected series and return its results.
`result_to_frame()` / `result_series_spec(s)()`	Turn a result into the DataFrame and series metadata (name, roles, style) the base class writes to the chart.
`format_results()`	Render the results as the text/HTML shown in the preview pane.
`refresh_results()`	Recompute and redraw the preview after a parameter changes.

Practical steps:

1. Create `app/series_operations/my_operation_dialog.py`.
2. Subclass `SeriesOperationDialogBase`, set `Name`, `Description`, `Icon`.
3. Declare parameters (e.g. with `FloatParam`/`IntParam`/`ChoiceParam`) and implement `build_parameter_selector`.
4. Implement `compute_results`, `result_to_frame`/`result_series_spec`, and `format_results`.
5. Restart the application — the operation appears in the Series Operations list.

`app/series_operations/function_dialog.py` is the smallest complete dialog and a reasonable starting template; `app/series_operations/calculus_dialog.py` is a good second read for an operation that consumes an existing series rather than generating one from scratch.